

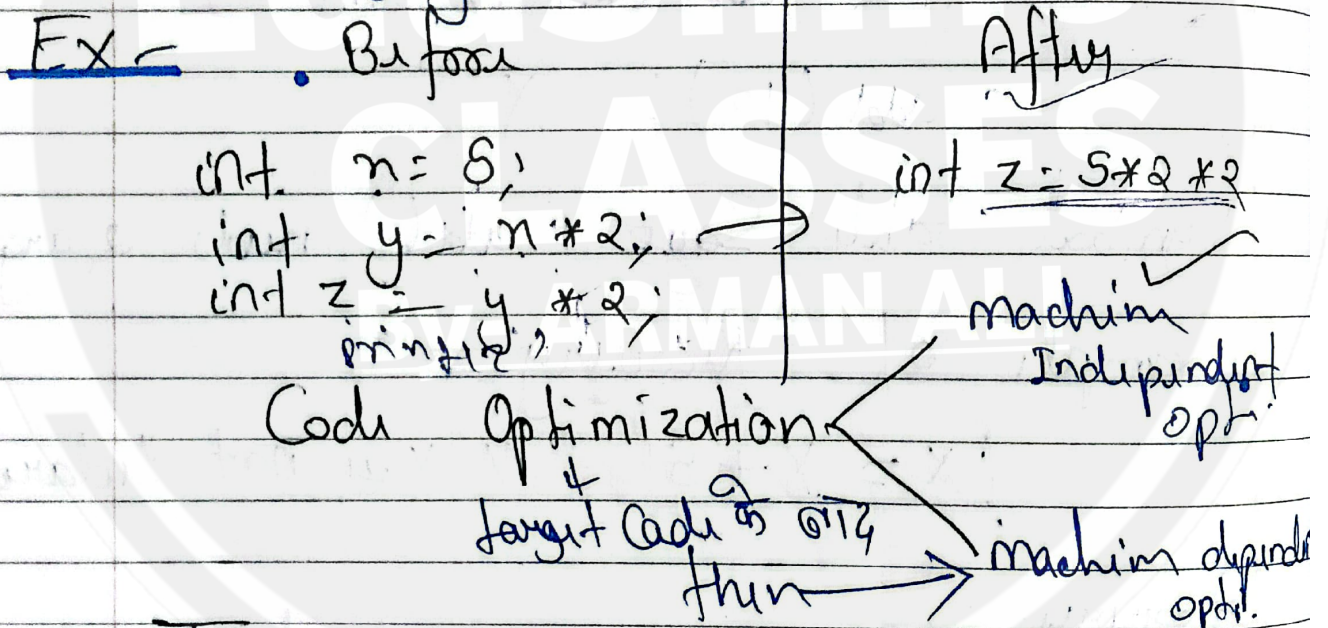
UNIT-5

Code Generation & Optimization

The Code Optimization is required to produce an efficient code.

The improvement over intermediate code by transformation is called Optimization.

→ Code Optimization means improving the quality of code without changing its o/p.



* Techniques used in Code Optimization -

① Basic blocks - Break intermediate code into blocks.

② Flow Graph - Graphical repⁿ of Basic block.

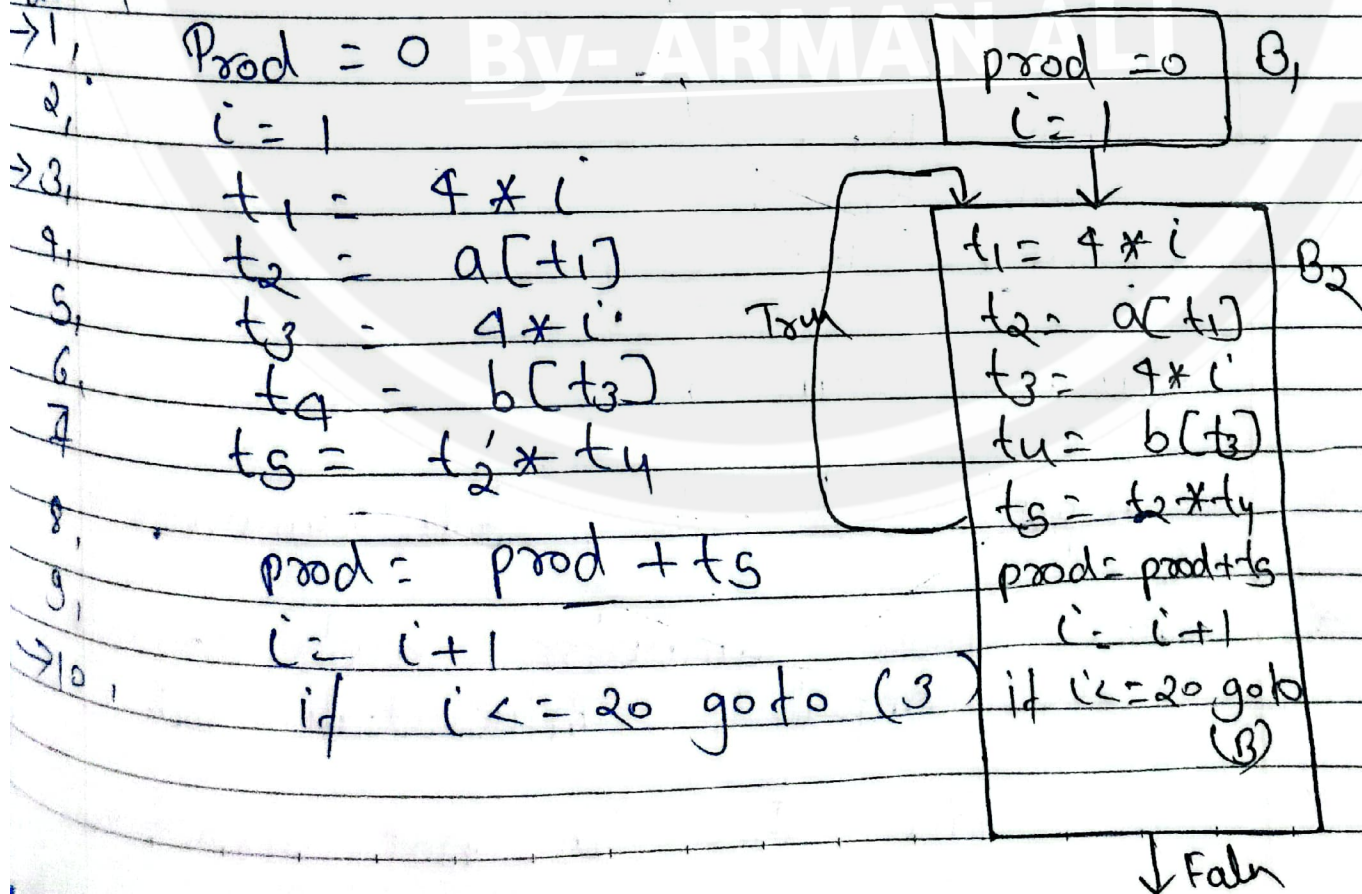
New Basic block

Convert following program into
three address code & identify
the basic blocks -

```

{ begin
  prod = 0;
  i = 1;
do
  begin
    prod = prod + a[i] * b[i]
    i = i + 1;
  end
while i <= 20
end;
  
```

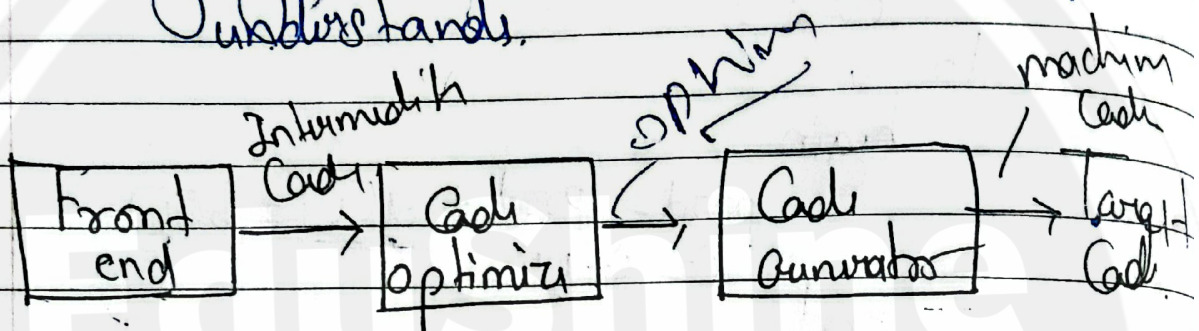
Thru address code — (Identify basic)



Code Generation

Code Generation is the final phase of the compiler.

It translates the optimized intermediate code or assembly language that the computer understands.



Ex- Intermediate Code

```
t1 = a + b
t2 = t1 * c
```

```
mov R1, a
ADD R1, b
MUL R1, c
```

Design issues

① Input to the Code Generator

- * The code generator needs correct & complete intermediate code.
- * if the I.C is not structured

will the final Code can be wrong or inefficient.

② Target Code Generation - The generated code should be -
Correct & efficient

③ Instruction Selection - Which instruction to use?

Ex. To add two no., we can use 'ADD' but which form
ADD, R₁, R₂ or ADD A, B

* The Goal is to choose fast & short instruction

④ Register Allocation - CPU has limited

* The no. of Register. The Code generator must use registers to avoid too many memory access.

⑤ Evaluation Order of Operands -

Which Operation to perform first. Quick

Ex - $a * b + c$
 ↑ ↑
 ① ② ✓

⑥ Code Siz. VS Code Speed -

Sometimes short code is not fastest and fast code is not the shortest. So compiler must balance. Space $\downarrow$ run $\uparrow$

* Basic block -

A basic block is a sequence of instructions which -

- The Flow of Control enters at the beginning and
- Leaves at the end without any halt or jump.
- There is no jump or branch instruction in the middle.

✓ A basic block is a straight-line code with no jump in or jump out in the middle.

✓

$a = b + c$
 $d = a - c$
`print (d)`

 } No jump or Condⁿ in between

* Algorithm to Convert TAC into blocks -

① Identify Leaders -

The First inst. is leader

Other Leaders are -

→ Any inst. that is the target of a jump.

→ Any inst. that comes after a jump

② Form Basic blocks -

For each

leader, make a basic block -

from that inst. to next leader -

Ex- Three address Code -

1, $a = b + c \rightarrow L_1$

1, 3, 6. Leaders

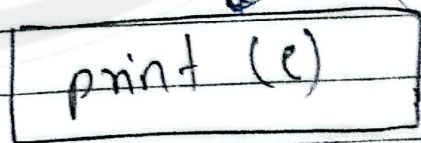
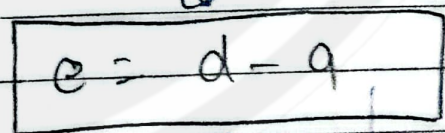
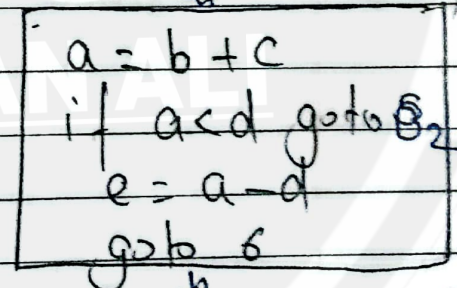
2, if $a < d$ goto 5

3, $e = a - d$

4, goto 6

5, $e = d - a \rightarrow L_2$

6, print (e) $\rightarrow L_3$



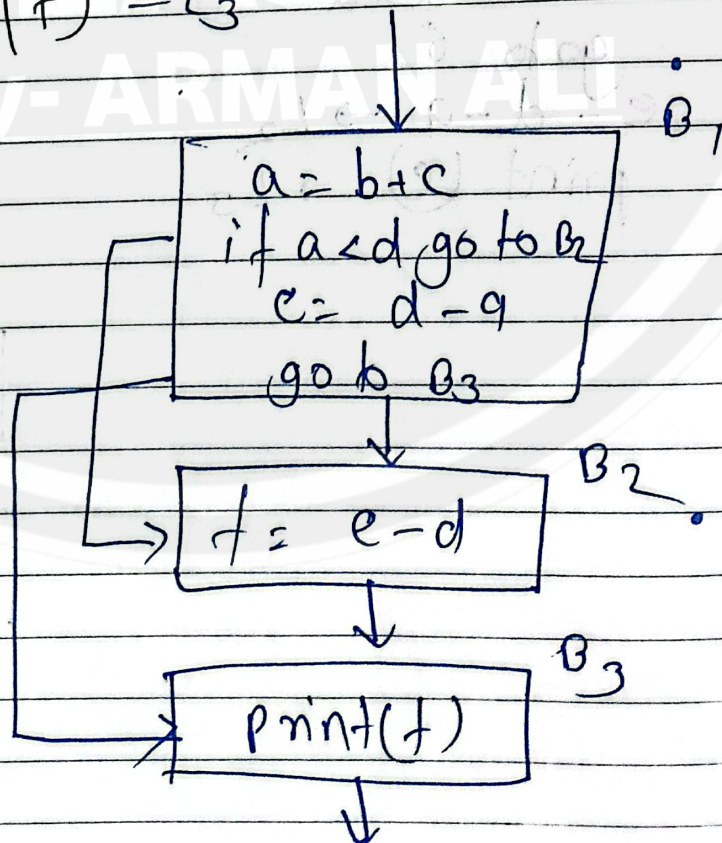
Flow Graph -

A flow graph is also called CFG, which shows how control flow between basic blocks in a program.

each node = a basic block

edge = flow of control

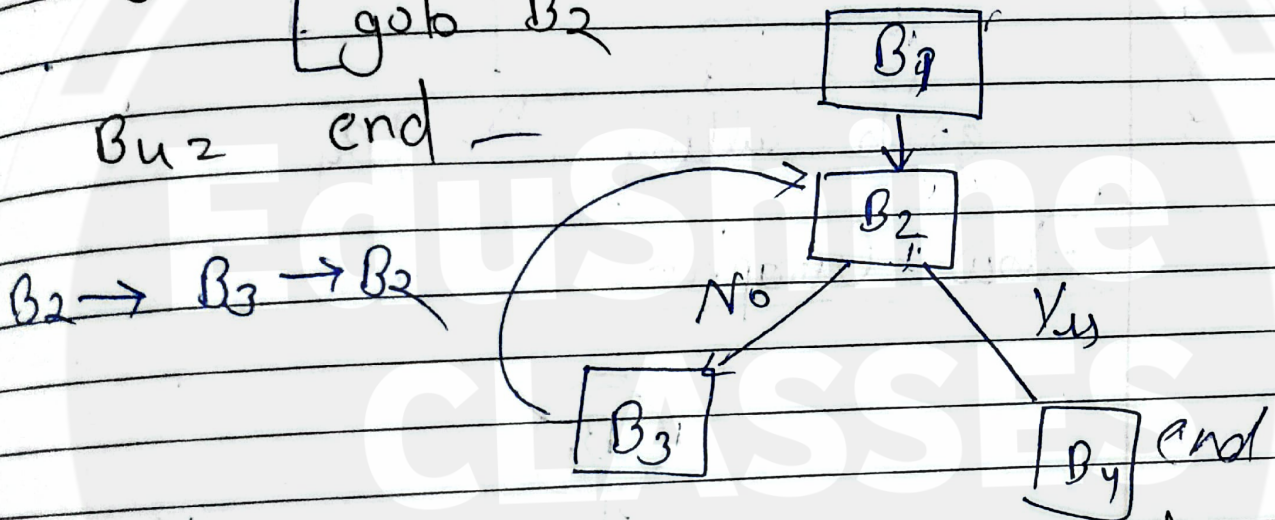
EX - 1,
1. $a = b + c$ - L₁
2. if $a < d$ goto 3
3. $e = d - a$
4. goto 6
5. $f = e - d$ - L₂
6. $\text{print}(f)$ - L₃



Loops in Flow Graph -

- when the control comes back to the previous block again.

Ex
 $B_1 - i = 0$
 $B_2 - \text{if } i > 10 \text{ goto } B_4$
 $B_3 = \begin{cases} i = i + 1 \\ \text{goto } B_2 \end{cases}$
 $B_4 - \text{end}$



Ques - Construct the flow graph for the following code segment -

```

{ fact(n)

```

```

    int f = 1
    for (i = 2; i ≤ n; i++)
        f = f * i
    return f;

```

```

}

```

Soln Thru oddnum Cdr -

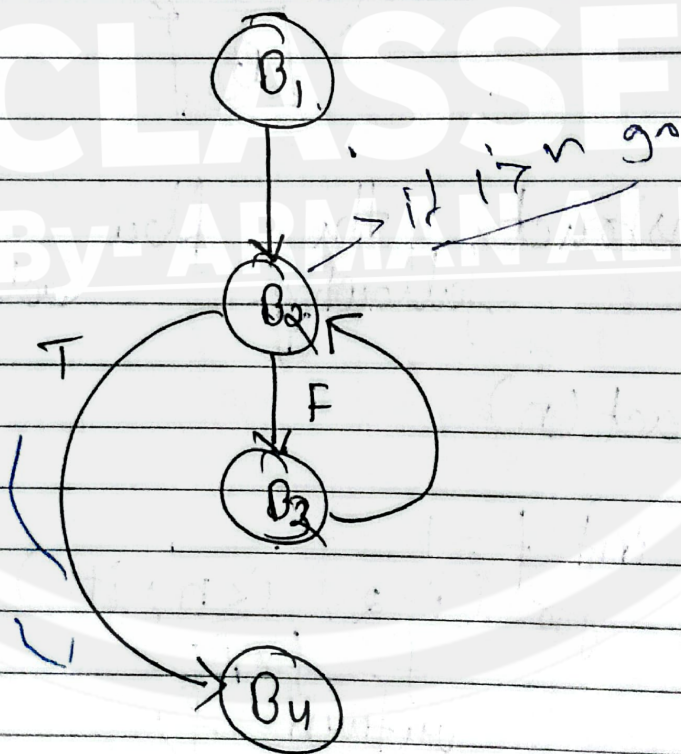
```

1,      t = 1
2,      i = 2
3,      if i > n go to 9
4,      t = t * i
5,      t = t / i
6,      t2 = i + 1
7,      i = t2
8,      go to (3)
9,      go to calling fn -

```

B₁
B₂
B₃
B₄

Flow Graph -



Optimization of basic block -

Improving the code inside the block -

Run faster ✓
Use less memory ✓
Remove useless steps ✓

Here, are common techniques to optimize:

① Constant Folding -

Do calculation at compile time.

Ex. $a = 4 * 5 \rightarrow a = 20$ ✓

② Constant Propagation -

Replace variable with its known value.

Ex $X = 10$;
 $Y = X + 5$; $\rightarrow Y = 10 + 5 \rightarrow Y = 15$ ✓

③ Common Subexpression elimination -

Remove repeated calculation -

Ex $a = b + c$
 $d = b + c \rightarrow d = a$ ✓

④ Dead Code Elimination - Remove Code that never affects the o/p.

Ex- $X = 10;$ → remove it's never used.
 $X = 20;$

⑤ Strength Reduction - Replace costly operations with cheaper ones.

Ex. $a = b * 2 \rightarrow a = b + b;$ ✓

⑥ Algebraic Simplification

$X = X + 0 \rightarrow X = X;$

$Y = Y * 1 \rightarrow Y = Y;$

Machine Independent Optimization

① Function pruning

optimization of basic block for

② Loop Optimization

Loop Optimization -

many improving the code
inside the loops to make
the program faster & efficient ✓

Here, are some common optimization techniques -

① Loop invariant Code motion -

the code outside the loop if
it doesn't change ^{→ move} inside.

EX-

```
{ for (i=0; i<10; i++)  
  {  
    x = 5*3 = 8  
    y = x*i;  
  }  
}
```

Optimize -

$x = 5 + 3$

```
for (i=0; i<10; i++) {
```

$y = x * i$
}

② Strength Reduction -

operations like multiplication
cheap ones like addition -
Replace expensive with

Exp- $\{ \text{for } (i=0; i<10; i++)$
 $\quad \quad \quad X = i*2;$
 $\quad \quad \quad \}$

Optimize -

$X = 0$
 $\{ \text{for } (i=0; i<10; i++)$
 $\quad \quad \quad X = X + 2;$
 $\quad \quad \quad \}$

③ Loop Unrolling -

of loop doing loop. control more Reduce the no. of statement work by

Ex- $\{ \text{for } (i=0; i<3; i++) \{$
 $\quad \quad \quad a[i] \geq 0;$
 $\quad \quad \quad \}$

Optimize -

$a[0] \geq 0$
 $a[1] \geq 0$ ✓
 $a[2] \geq 0$
 $a[3] \geq 0$

④ Loop fusion -

loops that work on the same data. Combine two adjacent do the same data.

** Pephali Optimization -

optimization is a small, local optimization technique used in Compilers.

You look at a small pephali of code (just a few instructions at a time) and try to make small improvements.

→ Compiler looks through a small window (pephali) at a few lines of machine code or even code at a time & replaces inefficient code with better code.

→ optimization of basic block of instructions

Directed Acyclic Graph -

* A DAG is used in Code Optimization (especially for expression) to -

- Remove common subexpression.
- Avoid repeating calculation.
- Represent expression in efficient form.

It is a graph that has

- ✓ Node - Represent variable (+, * --)
- ✓ Edges - Show the flow of data betⁿ operation.
- ✓ No Cycle - You can't come back to a node once you leave it -

Exp-
 $a = b + c$
 $d = b + c$

Without DAG, if sums like we should calculate $b+c$ twice.

but with DAG -

- $b+c$ is computed once. Stored in a node.
- both a & d point to the same.

This avoids duplicate work. ✓

*

Algorithm to Construct DAG -

① Start with an Empty DAG.

② For each statements -

a) check if Operands (like, b, c already exist in DAG)

Yes $\rightarrow$ use the existing node
No $\rightarrow$ create a new node.

b) check if operation like + with those operands already exists -

Yes $\rightarrow$ reuse it.
No $\rightarrow$ create a new node for it.

③ Assign the result variable

Ex. Construct DAG for $(a+b) * (a+b+c)$

Solⁿ

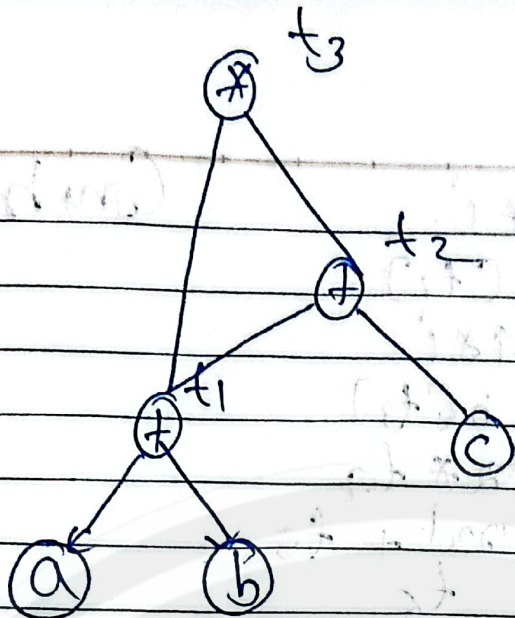
TAC -

$$t_1 = a + b$$

$$t_2 = t_1 + c$$

$$t_3 = t_1 * t_2$$

DAG



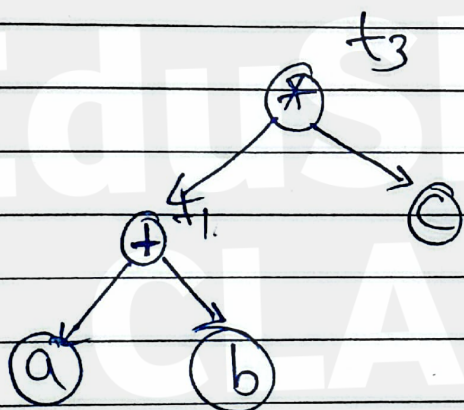
leaves
are
either
variable
or
constant

Ex-2

$$t_1 = a + b$$

$$t_2 = a + b$$

$$t_3 = t_1 * c$$



**

Ques

$$P + p * (q - r) + (q - r) * s$$

Solⁿ

TAC

$$t_1 = q - r$$

$$t_2 = t_1 * s$$

$$t_3 = p * t_1$$

$$t_4 = p + t_3$$

$$t_5 = t_4 + t_2$$

Unit - 5 Complete ✓



[Subscribe - EduShine Classes]